SCALE FOR PROJECT MALLOC (/PROJECTS/42CURSUS-MALLOC)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42paris.fr:vogsphere/intra-uuid-ab7c33f3-2bc3-492e-bee
```

Introduction

Please respect the following rules:

- Remain polite, courteous, respectful and constructive throughout the correction process. The well-being of the community depends on it.

- Identify with the person (or the group) graded the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

You MUST run the requested tests.

Warning: This project is quite complex, the result and its implementation are subjective. You have to keep in mind the aim of this project:

"This project is about implementing a dynamic memory allocation mechanism."

Attachments

test4.c (https://cdn.intra.42.fr/document/document/42708/test4.c)

test3.c (https://cdn.intra.42.fr/document/document/42709/test3.c)

run_linux.sh (https://cdn.intra.42.fr/document/document/42710/run_linux.sh)

test2.c (https://cdn.intra.42.fr/document/document/42711/test2.c)

test0.c (https://cdn.intra.42.fr/document/document/42712/test0.c)

test5.c (https://cdn.intra.42.fr/document/document/42713/test5.c)

test1.c (https://cdn.intra.42.fr/document/document/42714/test1.c)

run_mac.sh (https://cdn.intra.42.fr/document/document/42715/run_mac.sh)

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/188851/en.subject.pdf)

Preliminaries

Preliminary tests

First check the following elements :

- There is something in the git repository
- A Makefile is present and has all the requested rules
- No cheating (unauthorized functions...)
- 2 globals are authorized: one to manage the allocations, and one to manage thread-safety

If an element of this list isn't respected, the grading ends.
Use the appropriate flag. You're allowed to debate some more
about the project, but the grading will not be applied.

☑ Yes ☒ No

Library compilation

First we will check that the compilation of the library does
generate the requested files by modifying HOSTTYPE:

```
$> export HOSTTYPE=Testing
$> make re
********
$> ln -s libft_malloc_Testing.so libft_malloc.so
$> ls -l libft_malloc.so
lrwxrwxrwx 1 ** ** ** ** ** ** libft_malloc.so -> libft_malloc_Testing.so
$>
```

Does the Makefile use HOSTTYPE to define the name of the library
(libft_malloc_$HOSTTYPE.so) and create a symbolic link
libft_malloc.so pointing to libft_malloc_$HOSTTYPE.so?

If that's not the case, the defense stops.

☑ Yes ☒ No

Functions export

Check with nm that the library does export the functions malloc,
free, realloc and show_alloc_mem.

```
$> nm -g libft_malloc.so
0000000000000000 T _free
0000000000000000 T _malloc
0000000000000000 T _realloc
0000000000000000 T_show_alloc_mem
                 U _mmap
                 U _munmap
                 U _getpagesize (getpagesize under OSX or sysconf(_SC_PAGESIZ
E) under linux)
                 U _write
$>
```

The functions exported by the library are marked with a T, the used ones with a U (addresses have been replaced by 0, they change from one library to the next, same as the order of the lines).

If the functions are not exported, defense stops.

☑ Yes ☒ No

Feature's testing

Please find the attached (MacOS X or Linux) script that will only modify the environment variables while you run a test program.

Malloc test

We are first going to make a first test program that does not use malloc, so that we have a base to compare to.

Use test0.c file attached in the scale
WARNING: If you are using a linux vm, make sure that you are using the time binary that you can get with apt (sudo apt install time), else you won't have access to the -v option.

MAC:

```
$> gcc -o test0 test0.c && /usr/bin/time -l ./test0
```

LINUX:

```
$> gcc -o test0 test0.c && /usr/bin/time -v ./test0
```

We will then add a malloc and write in each allocation to make sure that the memory page is allocated in physical memory by MMU. The system will only really allocate the memory of a page if you write in it, so even if we do a bigger mmap than the malloc request it won't modify the "page reclaims".

For Linux => Major, Minor

For Mac OS X => page reclaims, page faults

Using the test1.c file given as attachment, run the following tests
MAC:

```
$> gcc -o test1 test1.c && /usr/bin/time -l ./test1
```

LINUX:

```
$> gcc -o test1 test1.c && /usr/bin/time -v ./test1
```

Our test1 program requested 1024 times 1024 bytes, so 1Mbyte. We can therefore check by doing the difference with the test0 program:

- either between the "maximum resident set size" lines, we obtain a little more than 1Mbyte
- or between the page reclaims lines that we will multiply by the value of getpagesize(3) under OSX or sysconf(_SC_PAGESIZE) under linux.

Let's test now both programs with our library:

MAC:

```
$>chmod 777 run_mac.sh

$>./run_mac.sh /usr/bin/time -l ./test0
  ********
  202  page reclaims
  0  page faults
  ********

$>./run_mac.sh /usr/bin/time -l ./test1
  ********
  525  page reclaims
  0  page faults
  ********
$>
```

LINUX:

```
$>chmod 777 run_linux.sh

$>./run_linux.sh /usr/bin/time -v ./test0
  ********
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 64
  ********

$>./run_linux.sh /usr/bin/time -v ./test1
  ********
  Major (requiring I/O) page faults: 0
  Minor (reclaiming a frame) page faults: 323
  ********
```

Count the number of pages used in addition to the real malloc and adjust the score accordingly:

- fewer pages than the real malloc, allocated memory is insufficient: 0
- 181 pages and over, malloc works but the overhead is way too big: 1

- between 91 pages and 180 pages, malloc works but the overhead is too big: 2
- between 51 pages and 90 pages, malloc works but the overhead is very big: 3
- between 21 and 50 pages more than real malloc, malloc works but the overhead is big: 4
- between 0 and 20 pages more than real malloc, malloc works and the overhead is fine: 5

Rate it from 0 (failed) through 5 (excellent)

Pre-allocated zones

Check inside the source code that the pre-allocated zones for the different malloc sizes allow to store at least 100 times the maximum size for this type of zone. Check also that the size of the zones is a multiple of getpagesize() under OSX or sysconf(_SC_PAGESIZE) under linux.

If one of these points is missing, click NO.

☑ Yes ☒ No

Tests of free

We will simply add a free to our test program:

```
$> cat test2.c
```

Now we will compare test2 with test1 to verify that free() works correctly:

- test1 allocates memory repeatedly without freeing it, forcing the allocator to request more pages from the system (high page faults).
- test2 allocates and then frees each block, allowing the allocator to reuse previously acquired memory instead of requesting more from the OS (low page faults).

If test2 has as many or more "page reclaims / page faults" than test1,
the free doesn't work.

MAC:

```
$> gcc -o test2 test2.c && ./run_mac.sh /usr/bin/time -l ./test2
```

LINUX:

```
$> gcc -o test2 test2.c && ./run_linux.sh /usr/bin/time -v ./test2
```

Does the free function? (less "page reclaims / page faults" than test1)

☑ Yes ☒ No

Quality of the free function

Now we will verify the quality of the free implementation by comparing test2 with test0:

- test0 is the baseline that doesn't use malloc at all.
- test2 uses malloc and free together.

If free() is implemented efficiently and properly reuses memory, test2

should have a similar number of page reclaims to test0 (the baseline).

Run test0 and test2. Test2 should not have more than 10 page reclaims compared to test0.

☑ Yes ☒ No

Realloc test

Using test3.c file given as attachment, test the following:

MAC:

```
$> gcc -o test3 test3.c -L. -lft_malloc && ./run_mac.sh ./test3
Hello world!
Hello world!
$>
```

LINUX:

```
$> gcc -o test3 test3.c -L. -lft_malloc && ./run_linux.sh ./test3
Hello world!
Hello world!
$>
```

The test must print out "Hello world!" two times.

Does it work as expected?

☑ Yes ☒ No

Show_alloc_mem test

Using test4.c file given as attachment, test the following:

MAC:

```
$> gcc -o test4 test4.c -L. -lft_malloc && ./run_mac.sh ./test4
$>
```

LINUX:

```
$> gcc -o test4 test4.c -L. -lft_malloc && ./run_linux.sh ./test4
$>
```

Does the display correspond to the subject and the TINY/SMALL/LARGE allocation of the project?

☑ Yes ☒ No

Alignment test

Using test5.c file given as attachment, test the following:

MAC:

```
$> gcc -o test5 test5.c -L. -lft_malloc && ./run_mac.sh ./test5
$>
```

LINUX:

```
$> gcc -o test5 test5.c -L. -lft_malloc && ./run_linux.sh ./test5
$>
```

You have no alignment errors.

☑ Yes ☒ No

Bonus

Competitive access

The project manages the competitive access of the threads with
the support of the pthread library and with mutexes.

Count the applicable cases:

- a mutex prevents multiple threads from simultaneously entering the malloc function
- a mutex prevents multiple threads from simultaneously entering the free function
- a mutex prevents multiple threads from simultaneously entering the realloc function
- a mutex prevents multiple threads from simultaneously entering the show_alloc_mem function

Rate it from 0 (failed) through 5 (excellent)

Additional bonuses

If there are more bonuses, grade them here. Bonuses must be 100%
functional and minimally useful. (up to the grader)

Bonus example:

- During a free, the project "defragments" the free memory while regrouping the available simultaneous blocks.
- Malloc has debugging environment variables.
- A function allows making a hexadecimal dump of the allocated zones.
- A function allows displaying a history of the memory allocations done.
- If there are other bonuses, add them here. Bonuses must be 100% functional and minimally useful (at the corrector's discretion).

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work No author file Invalid compilation Norme Cheat

Crash Forbidden function Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation